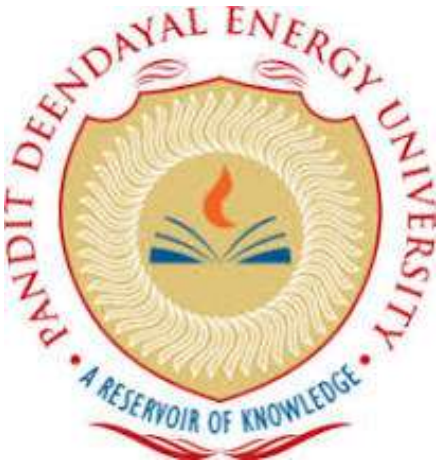
HAND GESTURE VOCALISER

IOT Project Report

Sensor-Based Hand Gesture Recognition using ESP32, Flex Sensors and MPU6500



Submitted by:	Chirag Maloo & Sumit Mali
Enrollment Number:	23BIT192 & 23BIT193
Department:	ICT – Div 3 [H9]
Submitted to:	Ms. Aarti Solanki
Academic Year:	July 2026 to November 2026

Department of Information and Communication Technology
School of Technology
PANDIT DEENDAYAL ENERGY UNIVERSITY, GANDHINAGAR, GUJARAT - 382426

1. ABSTRACT

The Hand Gesture Vocaliser is a low-cost, glove-based system that recognises a predefined set of hand gestures and converts them into text or symbols. Five flex sensors, one for each finger, measure finger bending, while an MPU6500 sensor provides three-axis accelerometer readings used to determine whether the hand is held UP or DOWN. An ESP32 DevKit V1 reads the five analog flex-sensor voltages and the accelerometer data and transmits them to a computer through a USB serial connection. A Python program processes the incoming readings, applies calibration-based thresholds to classify each finger as Straight or Bent, determines the hand orientation and combines multiple readings using majority voting. The resulting five-character finger sequence and the orientation are looked up in a gesture mapping table to obtain the output text or symbol. The system recognises only predefined gestures and is not a complete sign language translator. Conversion of the recognised output into speech is identified as a future extension and is not part of the current implementation.

2. INTRODUCTION

A hand gesture recognition system converts the configuration and orientation of a hand into digital information that a computer can interpret. Hand gestures are natural and need no keyboard or touch screen, so they can serve as an input method for communication and control applications. Sensor-based systems capture a gesture directly from the hand and do not depend on cameras or lighting conditions.

In this project, finger bending is captured by five flex sensors and hand orientation by the accelerometer of an MPU6500. A microcontroller samples the sensors and sends the readings to a computer, where software classifies them and maps the result to text or a symbol. The objective is to develop a low-cost, sensor-based prototype that recognises a predefined set of gestures.

3. PROBLEM STATEMENT AND OBJECTIVES

3.1 Problem Statement

Detecting a predefined hand gesture electronically requires measuring the bending of every finger and the orientation of the hand, and converting these physical quantities into a consistent digital representation. Raw sensor values vary with sensor characteristics, glove fit and electrical noise, so they cannot be used directly; they must be calibrated, classified and mapped to a meaningful output.

3.2 Objectives

1. To detect the bending of all five fingers.
2. To measure hand orientation using an accelerometer.
3. To acquire sensor readings using an ESP32.
4. To transmit sensor data to a computer through serial communication.
5. To process and classify sensor readings using Python.
6. To calibrate the system for different finger positions and hand orientations.
7. To map recognised gestures to corresponding text or symbols.
8. To provide a foundation for future speech output.

4. HARDWARE COMPONENTS

Sr.	Component	Quantity	Purpose
1	ESP32 DevKit V1	1	Reads the five analog sensor voltages and the MPU6500 data and sends them to the computer over USB serial.

Sr.	Component	Quantity	Purpose
2	Flex Sensors	5	One sensor per finger; resistance changes with the bending of the finger.
3	MPU6500	1	Measures acceleration along three axes; used to determine hand orientation (UP or DOWN).
4	10 kΩ Resistors	5	One fixed resistor per flex sensor, forming an individual voltage-divider circuit.
5	Breadboard	1	Provides temporary circuit connections without soldering.
6	Jumper Wires	As required	Connect the sensors, resistors and the ESP32.
7	Glove	1	Holds the sensors in position on the fingers and the hand.
8	USB Cable	1	Provides power and serial communication between the ESP32 and the computer.

5. SOFTWARE AND TECHNOLOGIES

Technology	Purpose / Status
Arduino IDE / ESP32 firmware	Reads the flex sensors (ADC) and the MPU6500 (I2C) and sends the readings over serial.
Python	Processes the readings: calibration, classification, majority voting and gesture mapping.
PySerial	Receives the serial data sent by the ESP32 in the Python program.
NumPy / other libraries	[Editable: list the Python libraries actually used]
React, Vite, FastAPI, WebSocket	Web interface components. [Editable: confirm implementation status before submission]

The recognition stage described in this report is threshold-based (calibration thresholds and majority voting); it is not a trained machine-learning classifier.

6. THEORETICAL BACKGROUND

6.1 Flex Sensor

A flex sensor is a thin, flexible strip whose electrical resistance changes when it is bent, so it behaves as a variable resistor. Bending generally increases its resistance, and the change follows the extent of bending. Mounting one sensor along each finger therefore gives a resistance that follows the bending of that finger. The ESP32 cannot read resistance directly because its analog inputs measure voltage; the resistance change must first be converted into a voltage change.

6.2 Voltage Divider Circuit

Each flex sensor is connected in series with a fixed 10 kΩ resistor between the supply and ground: 3.3 V → Flex Sensor → Junction → 10 kΩ Resistor → GND. The junction is connected to an ESP32 ADC input, and its voltage is

$$V_{out} = V_{in} \times R_2 / (R_1 + R_2)$$

where V_{in} = 3.3 V is the supply, R_1 is the flex sensor resistance, R_2 = 10 kΩ is the fixed resistor and V_{out} is the junction voltage measured by the ESP32. When a finger bends, R_1 changes and V_{out} changes with it; for this connection, an increase in R_1 lowers V_{out}. Every finger has its own sensor, its own 10 kΩ resistor and its own ADC input.

6.3 Analog-to-Digital Conversion (ADC)

An analog signal varies continuously, whereas a digital signal is a discrete number. The ADC converts the junction voltage into an integer reading that increases approximately with the input voltage. An n-bit ADC gives readings from 0 to $2^n - 1$, so a 12-bit ADC has a theoretical range of 0–4095. This range is only theoretical: the actual values depend on the circuit, the sensor resistance and the ADC configuration, which is why calibration is required.

6.4 MPU6500 Inertial Measurement Unit

An Inertial Measurement Unit (IMU) measures quantities related to motion. The MPU6500 contains a three-axis accelerometer, which measures acceleration along the X, Y and Z axes including the effect of gravity, and a three-axis gyroscope, which measures angular rate. When the sensor is stationary or moving slowly, gravity dominates, so the way gravity is shared among the X, Y and Z readings depends on the orientation of the sensor. Turning the hand between UP and DOWN changes the gravity vector relative to the sensor, and the accelerometer readings change accordingly, which allows the two orientations to be distinguished. The current approach uses only the accelerometer X, Y and Z readings; gyroscope readings are not used in recognition.

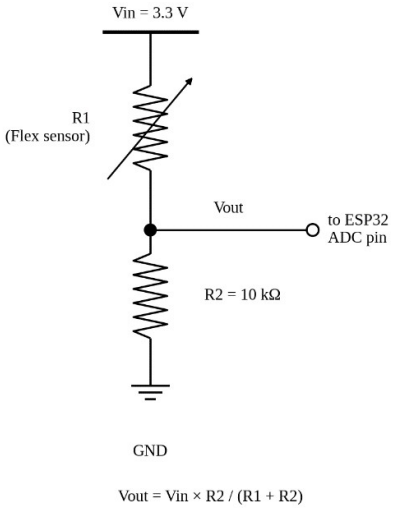
6.5 I2C Communication

I2C is a two-wire serial bus on which devices share the SDA (Serial Data) and SCL (Serial Clock) lines. It suits the MPU6500 because only two signal lines are needed to read its data. The ESP32 acts as the I2C master and generates the clock; the MPU6500 acts as the slave. Each slave has an I2C address, which the master sends to select the device it wants to communicate with.

6.6 ESP32 Microcontroller

A microcontroller is a single integrated circuit containing a processor, memory and input/output peripherals. The ESP32 is used because one low-cost board provides analog inputs, I2C and USB serial communication. In this project it reads five analog flex-sensor signals, communicates with the MPU6500 through I2C and transmits the eight readings (five flex values and three accelerometer values) to the computer.

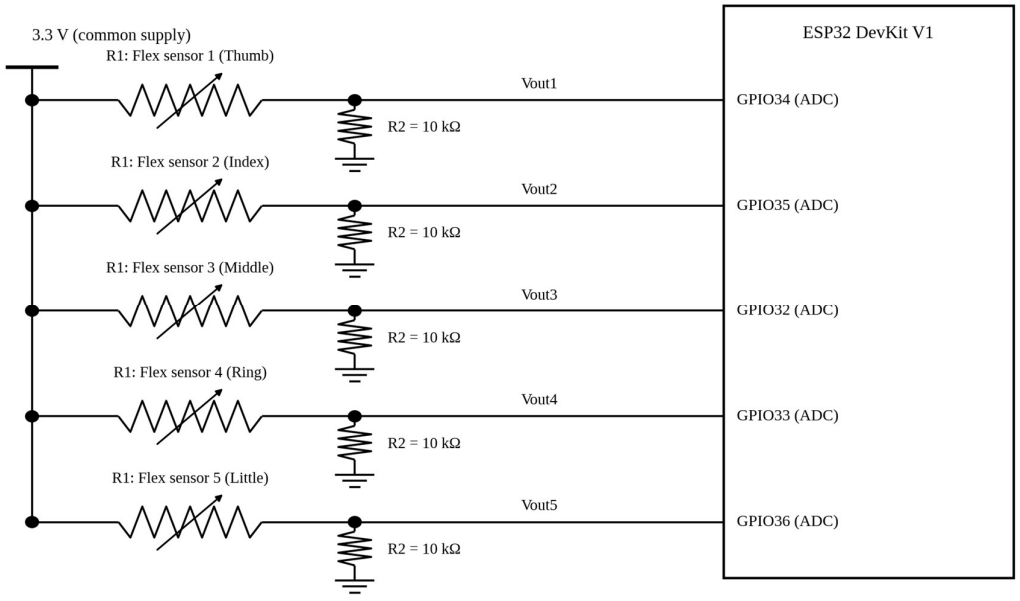
7. CIRCUIT DIAGRAMS



Finger	ESP32 GPIO
Thumb	GPIO 34
Index	GPIO 35
Middle	GPIO 32
Ring	GPIO 33
Little	GPIO 36

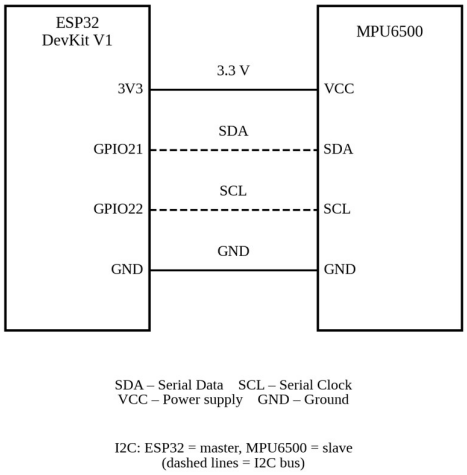
Figure 7.1: Flex Sensor Voltage Divider Circuit.

Figure 7.1 shows one voltage divider: R_1 is the flex sensor, R_2 = 10 kΩ, V_{in} = 3.3 V, and the junction voltage V_{out} is read by an ESP32 ADC pin. The same circuit is repeated independently for all five fingers (Figure 7.2) using the GPIO assignment below.



Each divider has its own R1, its own R2 and its own ADC input. All GND symbols are the common ESP32 GND.

Figure 7.2: Five-Finger Flex Sensor Interfacing with ESP32.



SDA – Serial Data SCL – Serial Clock
VCC – Power supply GND – Ground

I2C: ESP32 = master, MPU6500 = slave
(dashed lines = I2C bus)

Figure 7.3: MPU6500–ESP32 I2C Interfacing.

MPU6500 Pin	ESP32 Connection
VCC	3.3 V
GND	GND
SDA	GPIO 21
SCL	GPIO 22

The MPU6500 is connected to the ESP32 as listed below. The ESP32 is the I2C master and the MPU6500 is the slave. Only the accelerometer X, Y and Z readings are used for recognition, and no connections are made to unused MPU6500 pins.

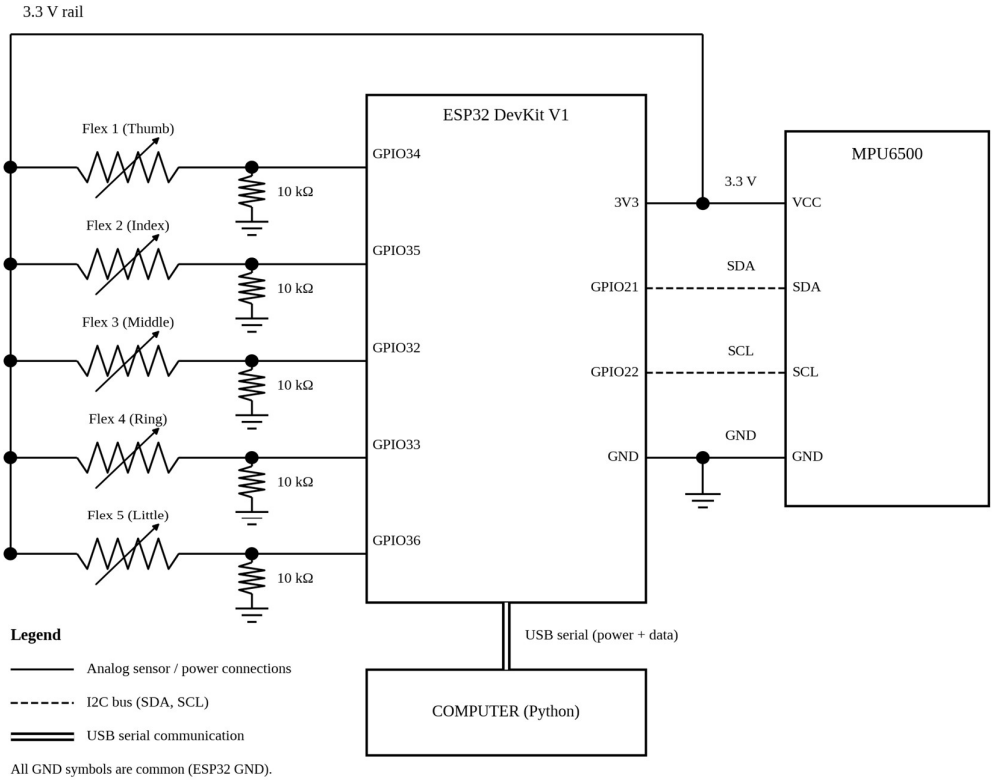


Figure 7.4: Complete Hardware Circuit Diagram of the Hand Gesture Vocaliser.

8. SYSTEM ARCHITECTURE

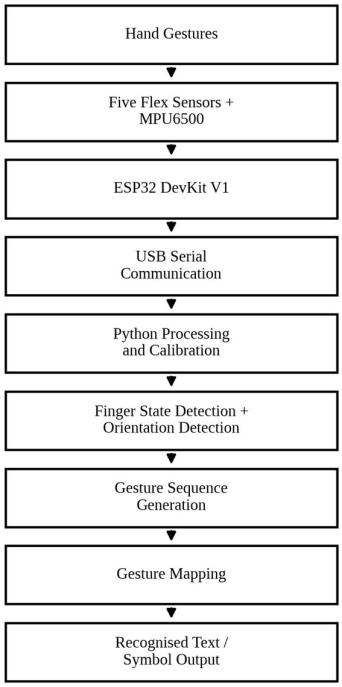


Figure 8.1: Overall System Architecture of the Hand Gesture Vocaliser.

1. **Hand gestures:** the user forms a predefined gesture.
2. **Sensors:** flex sensors measure finger bending; the MPU6500 measures acceleration.
3. **ESP32:** samples the five analog voltages and reads the accelerometer over I2C.
4. **USB serial:** carries the eight readings to the computer.
5. **Python processing and calibration:** receives the readings and applies the calibration thresholds.
6. **State detection:** each finger is classified S or B and the orientation U or D.
7. **Sequence generation:** majority voting over several readings gives a five-character sequence.
8. **Gesture mapping:** the sequence and orientation are looked up in the mapping table.
9. **Output:** the corresponding text or symbol is displayed.

9. WORKING PRINCIPLE

1. The user wears the glove carrying the five flex sensors.
2. Each flex sensor responds to the bending of its finger.
3. The MPU6500 measures acceleration along the X, Y and Z axes.
4. The ESP32 reads the five analog flex-sensor voltages through its ADC.
5. The ESP32 reads the MPU6500 accelerometer values through I2C.
6. The ESP32 sends the sensor readings to the computer over USB serial.
7. Python receives and processes the incoming readings.
8. Each finger reading is classified as Straight (S) or Bent (B).
9. The accelerometer readings classify the orientation as UP (U) or DOWN (D).
10. Multiple readings are collected during a recognition window.
11. Majority voting determines the final finger states and orientation.
12. The finger states form a five-character gesture sequence.
13. The sequence and orientation are looked up in the gesture mapping table.
14. The corresponding text or symbol is displayed.

10. CALIBRATION PROCESS

Calibration is necessary because raw flex readings differ with sensor resistance, sensor placement, glove fit, finger positions and electrical conditions, so fixed values would not suit every setup. The system collects sensor readings in four calibration stages:

Stage	Meaning
S	All fingers held straight
B	All fingers held bent
U	Hand held in UP orientation
D	Hand held in DOWN orientation

10.1 Flex Sensor Calibration

The readings collected in the S and B stages are used to determine, for each finger, a threshold that separates the straight state from the bent state. A live reading is then compared with this threshold to decide S or B. [Editable: insert the exact threshold formula and comparison direction from the code.] The thumb follows a special rule in the current implementation: a reading equal to zero is Straight (S) and a non-zero reading is Bent (B).

10.2 Orientation Calibration

The MPU6500 provides acceleration X, Y and Z. The readings collected in the U and D stages serve as a reference for distinguishing the two orientations, and a live accelerometer reading is compared with this reference to decide U or D. [Editable: insert the exact axis and threshold computation from the code.]

11. GESTURE RECOGNITION AND MAJORITY VOTING

The state of each finger is S or B, so the thumb, index, middle, ring and little fingers together form a five-character sequence. For example, S B S S B means thumb straight, index bent, middle straight, ring straight and little bent. The orientation is U or D, and the pair (sequence, orientation) identifies a gesture.

A single reading may be unstable because of sensor fluctuations, slight finger movements, electrical noise and inconsistent hand position. Therefore, multiple readings are collected in a recognition

window [Editable: confirm duration, number of samples and tie-handling rule], each reading is classified, and the most frequent state of each finger and the most frequent orientation are selected. Majority voting reduces the effect of unstable readings but does not guarantee error-free recognition. The table below is an illustrative example only, not measured data.

Reading	Thumb	Index	Middle	Ring	Little	Orientation
1	S	S	B	B	S	U
2	S	S	B	B	S	U
3	B	S	B	S	S	D
Final (majority)	**S**	**S**	**B**	**B**	**S**	**U**

12. GESTURE MAPPING TABLE

Sr. No.	Finger Sequence	Orientation	Mapped Gesture / Text
1	SBBBB	U	A
2	SBBBB	D	N
3	SSBBB	U	B
4	SSBBB	D	O
5	SSSSS	U	C
6	SSSSS	D	P
7	BSSSS	U	D
8	BSSSS	D	Q
9	BSBBB	U	E
10	BSBBB	D	R
11	SSSBS	U	F
12	SSSBS	D	S
13	SSSBB	U	J
14	SSSBB	D	T
15	SBBBS	U	H
16	SBBBS	D	U
17	BBBBS	U	I
18	BBBBS	D	V
19	BSSBB	U	J
20	BSSBB	D	W
21	BBSSS	U	K
22	BBSSS	D	X
23	SSBBS	U	L
24	SSBBS	D	Y
25	BBBBB	U	M

Sr. No.	Finger Sequence	Orientation	Mapped Gesture / Text
26	BBBBB	D	Z

Note: each gesture is identified by combining a five-finger state sequence (S = Straight, B = Bent; thumb to little finger) with the hand orientation (U = UP, D = DOWN). The 26 entries correspond to 13 finger sequences, each in two orientations. The Finger Sequence and Mapped Gesture / Text cells are to be filled in from the project's gesture mapping.

13. SOFTWARE IMPLEMENTATION

Python communicates with the ESP32 through PySerial and receives the sensor values. It applies the calibration thresholds to detect the finger states, processes the accelerometer readings to determine the orientation, forms the final finger sequence by majority voting and performs the gesture mapping. The classification is threshold-based and does not use a trained machine-learning model. A web interface may provide calibration and gesture-reading controls, sensor and connection status, and display of the recognised output. [Editable: implementation status – confirm that the web interface is implemented and integrated before submission.]

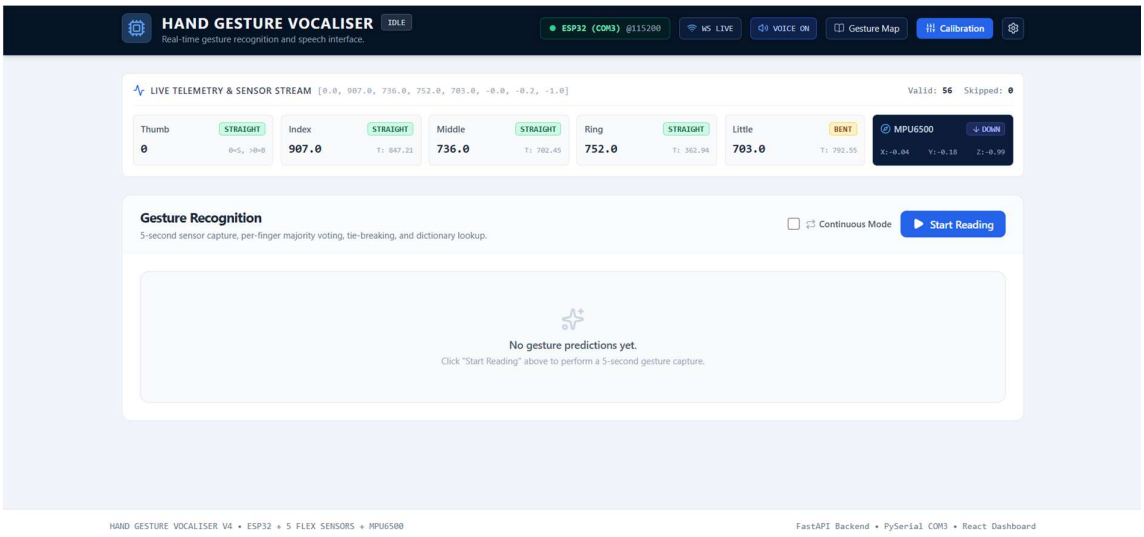


Figure 13.1: Main web dashboard.

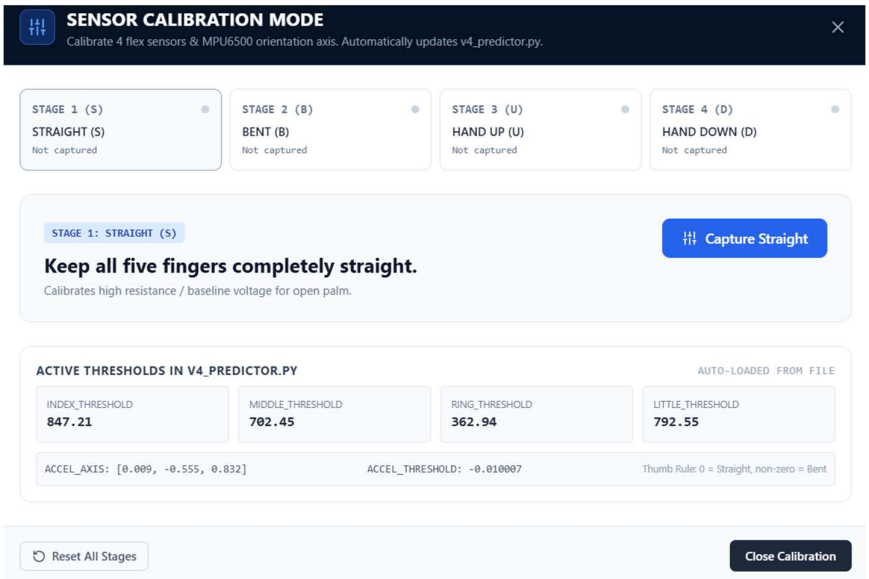


Figure 13.2: Calibration interface.

14. HARDWARE IMPLEMENTATION

The flex sensors are mounted along the fingers of the glove, and the MPU6500 is mounted on the back (centre) of the hand. The sensors are wired to the ESP32 through the breadboard as shown in Figure 7.4, and the ESP32 is connected to the computer through a USB cable. No accuracy or performance figures are reported in this document.

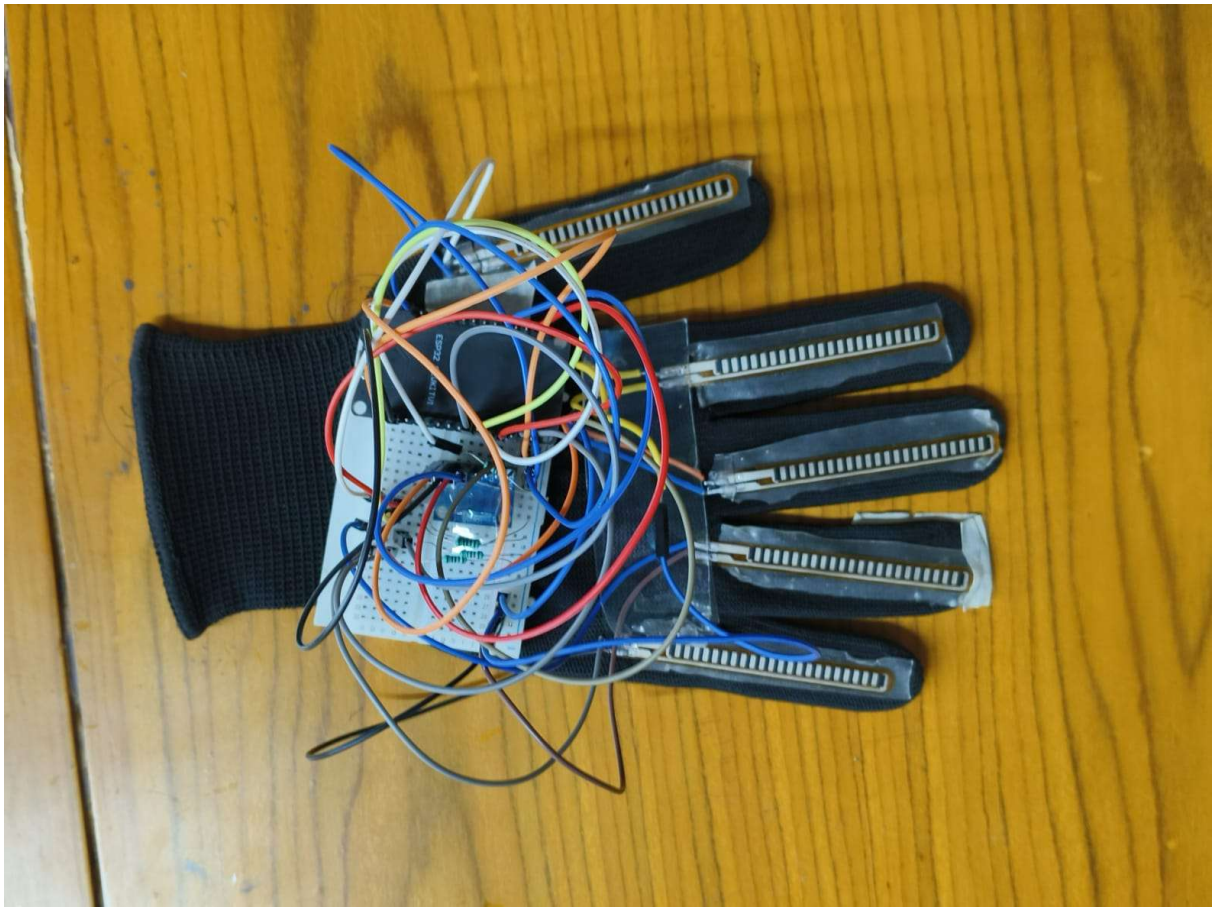


Figure 14.1: Complete hardware setup.

15. ADVANTAGES AND LIMITATIONS

Advantages	Limitations
• Uses commonly available hardware. • Captures finger bending and hand orientation. • Provides a modular sensor-based approach. • Allows calibration for sensor variation. • Can be extended with additional gestures. • Uses a removable glove arrangement.	• Flex sensor readings vary with placement and bending. • Calibration is required. • Sensor noise may affect recognition. • Only predefined gestures can be identified. • Depends on a wired USB serial connection. • Performance depends on calibration quality and sensor positioning.

16. FUTURE SCOPE

The following improvements are proposals and are not part of the completed implementation: adding more gesture combinations and a larger gesture vocabulary; improving calibration and noise filtering; a more compact and portable hardware design; wireless communication between the ESP32 and the computer; text-to-speech output; machine-learning-based gesture recognition; and improved reliability under different hand positions.

17. CONCLUSION

A sensor-based Hand Gesture Vocaliser has been developed in which five flex sensors detect finger bending and an MPU6500 accelerometer detects hand orientation. The ESP32 acquires the sensor data and sends it to a computer, where Python applies calibration thresholds, classifies finger states and orientation, and uses majority voting to obtain a stable result. The five-character finger sequence and the orientation are mapped to text or symbols through a predefined gesture table. Speech output remains a possible future extension. No accuracy or performance claims are made in this report.